Day 15 Task Workflow — Wednesday, Mar 26, 2026

> **Sprint:** 5 (Feature Depth) | **Phase:** Reports Module + Alert IOC Enrichment + IF Retrain + 100% API Coverage
> **Owner:** Lead Architect | **Status:** ⚪ Ready to Start
> **Goal:** Reports module (3 endpoints), alert IOC enrichment, IF retrain execution, system config → 42/42 (100% API coverage)
> **Grade:** Week 4 Day 1 COMPLETE ☑ | Week 4 Day 2 STARTING ⚫

Day 14 Results Context

100% VPS verification pass rate on all 7 tasks:

```
Day 14 Achievements:
  ☑ Threat Intel API Keys: All 3 providers enabled (OTX, AbuseIPDB, VirusTotal)
  ☑ OTX Sync: 1,367 IOCs populated (720 hash, 480 domain, 114 url, 53 ip)
  ☑ §11.3 Item 2: check_domain() + c2_phishing flag working
  ☑ §11.3 Item 3: VirusTotalClient + check_hash() + malware flag working
  ☑ Live IOC Correlation: 6/6 test cases passed
  ☑ Tuned IF Params: c=0.10, ms=1024 applied in hyperparams.py
  ☑ PCAP Upload: POST /capture/upload-pcap operational
  ☑ Bug fixes: ON CONFLICT 3-col match, tags list type for text[]

v0.4.0 Critical MVP: ACHIEVED ☑
  Pipeline: capture → ML → alerts → IOC (IP+domain+hash) → LLM narrative →
WebSocket
  IOC Database: 1,367 indicators (Silver Fox APT detected)
  §11.3 Correlation: FULLY COMPLIANT (3/3)
  API Coverage: 38/42 (90.5%)

Container Status (Day 14 Final):
  tm-backend    ☑ Rebuilt Day 14
  tm-capture    ☑ Up 4+ days
  tm-ml-worker  ☑ Rebuilt Day 14 (tuned IF params)
  tm-postgres   ☑ Healthy 5 days
  tm-redis      ☑ Healthy 5 days
```

Scope Adherence Checklist

Requirement	Source Document	Section	Status
Report PDF generation	MASTER_DOC_PART2	§5.1	❌ **NOT DONE — DO TODAY**
Report listing	MASTER_DOC_PART2	§5.1	❌ **NOT DONE — DO TODAY**
Report download	MASTER_DOC_PART2	§5.1	❌ **NOT DONE — DO TODAY**

Requirement	Source Document	Section	Status
System config endpoint	MASTER_DOC_PART2	§5.1	❌ **NOT DONE — DO TODAY**
Alert IOC enrichment	MASTER_DOC_PART4	§11.3	⚠️ Alerts don't return IOC match data
IF retrain execution	MASTER_DOC_PART4	§4.4	⚠️ Params in code, model not regenerated
CICIDS2017 validation	MASTER_DOC_PART4	§3	⏳ Optional for Day 15
Ensemble weights (0.30/0.45/0.25)	MASTER_DOC_PART4	§1.2	🔒 LOCKED
Alert thresholds (0.90/0.75/0.50/0.30)	MASTER_DOC_PART4	§1.2	🔒 LOCKED

LOCKED CONSTRAINTS (DO NOT MODIFY)

```
Ensemble Weights:
  composite = 0.30 × IF + 0.45 × RF + 0.25 × AE

Alert Thresholds:
  ≥ 0.90 → CRITICAL
  ≥ 0.75 → HIGH
  ≥ 0.50 → MEDIUM
  ≥ 0.30 → LOW
  < 0.30 → NONE

LLM Provider: OpenRouter only (3 verified models)
DO NOT suggest: Kafka, Kubernetes, Elasticsearch, MongoDB
```

Day 15 Objective

By end of day:

- Reports module fully operational (3 API endpoints: generate, list, download)
- System config endpoint implemented (GET /system/config)
- **API coverage: 42/42 = 100%** 🎯
- Alert API responses enriched with IOC correlation data
- IF model retrained with tuned params (c=0.10, ms=1024) — new model artifact produced
- Old test alerts cleaned up

> **NOTE:** All manual testing and VPS verification performed by Lead Architect.

Task Breakdown

TASK 1 — Reports Module: 3 API Endpoints 🔴

Time Est: 90 min | **Priority:** 🔴 Critical — Completes reports module (0/3 → 3/3)
Source: MASTER_DOC_PART2 §5.1, MASTER_DOC_PART3 §10

1.1 Create `backend/app/api/v1/reports.py`

Per Part 2 §5.1, the Reports service needs:

Method	Endpoint	Function
POST	`/reports/generate`	Generate PDF report
GET	`/reports/`	List generated reports
GET	`/reports/{id}/download`	Download PDF

Per Part 3 §10.1, report types include:

- **Daily Threat Summary** — 24h alerts, top threats, anomaly trends, AI narrative
- **Incident Report** — Per-alert details, timeline, affected IPs
- **Executive Briefing** — High-level threat posture, risk score
- **ML Performance Report** — Model comparison, accuracy trends
- **Network Health Report** — Traffic patterns, protocol distribution
- **Compliance Report** — Alert response times, resolution rates

Implementation:

```
"""
ThreatMatrix AI — Reports API

Per MASTER_DOC_PART2 §5.1:
  POST /reports/generate        → Generate report (async)
  GET  /reports/                → List generated reports
  GET  /reports/{id}/download   → Download PDF
"""

from __future__ import annotations

import json
import logging
import uuid
from datetime import datetime, timezone
from typing import Optional

from fastapi import APIRouter, HTTPException, Query
from fastapi.responses import FileResponse
from pydantic import BaseModel
from sqlalchemy import text

from app.database import async_session
```

```python
logger = logging.getLogger(__name__)

router = APIRouter(prefix="/reports", tags=["Reports"])


class ReportRequest(BaseModel):
    """Report generation request."""
    report_type: str  # daily_summary, incident, executive, ml_performance,
network_health, compliance
    title: Optional[str] = None
    date_range_start: Optional[str] = None  # ISO 8601
    date_range_end: Optional[str] = None
    alert_id: Optional[str] = None  # For incident reports


@router.post("/generate")
async def generate_report(request: ReportRequest):
    """
    Generate a report (async).
    Per MASTER_DOC_PART3 §10.1: 6 report types supported.

    Reports are generated as JSON summaries. PDF generation (ReportLab) is Week 6.
    """
    report_id = str(uuid.uuid4())
    now = datetime.now(timezone.utc)

    # Gather data based on report type
    report_data = {}

    async with async_session() as session:
        if request.report_type in ("daily_summary", "executive"):
            # Alert statistics (last 24h)
            alert_stats = await session.execute(
                text("""
                    SELECT severity, COUNT(*) as cnt
                    FROM alerts
                    WHERE created_at >= NOW() - INTERVAL '24 hours'
                    GROUP BY severity
                    ORDER BY cnt DESC
                """)
            )
            report_data["alert_stats"] = [
                {"severity": r[0], "count": r[1]} for r in alert_stats.fetchall()
            ]

            # Total flows (last 24h)
            flow_count = await session.execute(
                text("""
                    SELECT COUNT(*),
                           COUNT(*) FILTER (WHERE is_anomaly = true),
                           AVG(anomaly_score) FILTER (WHERE anomaly_score > 0)
                    FROM network_flows
                    WHERE created_at >= NOW() - INTERVAL '24 hours'
```

```
                """)
            )
            row = flow_count.fetchone()
            report_data["flow_stats"] = {
                "total_flows": row[0] if row else 0,
                "anomalous_flows": row[1] if row else 0,
                "avg_anomaly_score": round(float(row[2] or 0), 4),
            }

            # IOC stats
            ioc_count = await session.execute(
                text("SELECT ioc_type, COUNT(*) FROM threat_intel_iocs WHERE
is_active = true GROUP BY ioc_type")
            )
            report_data["ioc_stats"] = [
                {"type": r[0], "count": r[1]} for r in ioc_count.fetchall()
            ]

        elif request.report_type == "ml_performance":
            # Model registry data
            models = await session.execute(
                text("SELECT name, model_type, metrics, status FROM ml_models
ORDER BY created_at DESC LIMIT 10")
            )
            report_data["models"] = [
                {"name": r[0], "type": r[1], "metrics": r[2], "status": r[3]}
                for r in models.fetchall()
            ]

        elif request.report_type == "incident" and request.alert_id:
            # Single alert details
            alert = await session.execute(
                text("SELECT * FROM alerts WHERE alert_id = :aid OR id::text =
:aid"),
                {"aid": request.alert_id},
            )
            row = alert.fetchone()
            if row:
                report_data["alert"] = dict(row._mapping) if hasattr(row,
'_mapping') else {"id": str(row[0])}

        elif request.report_type == "network_health":
            # Protocol distribution
            protos = await session.execute(
                text("""
                    SELECT protocol, COUNT(*) as cnt
                    FROM network_flows
                    WHERE created_at >= NOW() - INTERVAL '7 days'
                    GROUP BY protocol
                    ORDER BY cnt DESC
                """)
            )
            report_data["protocol_distribution"] = [
                {"protocol": r[0], "count": r[1]} for r in protos.fetchall()
```

```
            ]

        # Persist report metadata
        await session.execute(
            text("""
                INSERT INTO system_config (key, value, description, updated_at)
                VALUES (:key, :value, :desc, :now)
                ON CONFLICT (key) DO UPDATE SET value = :value, updated_at = :now
            """),
            {
                "key": f"report:{report_id}",
                "value": json.dumps({
                    "id": report_id,
                    "report_type": request.report_type,
                    "title": request.title or f"{request.report_type.replace('_',
' ').title()} Report",
                    "status": "complete",
                    "data": report_data,
                    "generated_at": now.isoformat(),
                    "format": "json",
                }),
                "desc": f"Generated report: {request.report_type}",
                "now": now,
            },
        )
        await session.commit()

    return {
        "id": report_id,
        "report_type": request.report_type,
        "title": request.title or f"{request.report_type.replace('_', '
').title()} Report",
        "status": "complete",
        "data": report_data,
        "generated_at": now.isoformat(),
    }


@router.get("/")
async def list_reports(
    limit: int = Query(20, ge=1, le=100),
    offset: int = Query(0, ge=0),
):
    """
    List all generated reports.
    Per MASTER_DOC_PART2 §5.1.
    """
    async with async_session() as session:
        result = await session.execute(
            text("""
                SELECT key, value, updated_at
                FROM system_config
                WHERE key LIKE 'report:%'
                ORDER BY updated_at DESC
```

```
                LIMIT :limit OFFSET :offset
            """),
            {"limit": limit, "offset": offset},
        )
        rows = result.fetchall()

        count_result = await session.execute(
            text("SELECT COUNT(*) FROM system_config WHERE key LIKE 'report:%'")
        )
        total = count_result.scalar()

    reports = []
    for r in rows:
        try:
            data = r[1] if isinstance(r[1], dict) else json.loads(r[1])
            reports.append({
                "id": data.get("id", r[0].replace("report:", "")),
                "report_type": data.get("report_type", "unknown"),
                "title": data.get("title", "Report"),
                "status": data.get("status", "complete"),
                "generated_at": data.get("generated_at", str(r[2])),
                "format": data.get("format", "json"),
            })
        except Exception:
            pass

    return {
        "reports": reports,
        "total": total,
        "limit": limit,
        "offset": offset,
    }


@router.get("/{report_id}/download")
async def download_report(report_id: str):
    """
    Download a generated report.
    Per MASTER_DOC_PART2 §5.1.

    Week 6 will add PDF generation via ReportLab.
    Currently returns JSON data.
    """
    async with async_session() as session:
        result = await session.execute(
            text("SELECT value FROM system_config WHERE key = :key"),
            {"key": f"report:{report_id}"},
        )
        row = result.fetchone()

    if not row:
        raise HTTPException(status_code=404, detail="Report not found")

    data = row[0] if isinstance(row[0], dict) else json.loads(row[0])
```

```
    return {
        "id": report_id,
        "report_type": data.get("report_type"),
        "title": data.get("title"),
        "data": data.get("data", {}),
        "generated_at": data.get("generated_at"),
        "format": "json",
        "note": "PDF generation available in Week 6 (ReportLab)",
    }
```

1.2 Register Router in main.py

Add to `backend/app/main.py`:

```
from app.api.v1.reports import router as reports_router
app.include_router(reports_router, prefix="/api/v1")
```

Verification:

#	Check	Expected
1	`POST /reports/generate {"report_type": "daily_summary"}`	200, report_id returned, alert_stats + flow_stats
2	`POST /reports/generate {"report_type": "ml_performance"}`	200, models list
3	`POST /reports/generate {"report_type": "network_health"}`	200, protocol_distribution
4	`POST /reports/generate {"report_type": "executive"}`	200, same as daily_summary
5	`GET /reports/`	List of generated reports with metadata
6	`GET /reports/{id}/download`	Full report data
7	`GET /reports/nonexistent/download`	404
8	OpenAPI docs show 3 new endpoints	Visible under "Reports" tag

TASK 2 — System Config Endpoint 🟤

Time Est: 20 min | **Priority:** 🟤 Critical — Last missing endpoint for 100% API coverage
Source: MASTER_DOC_PART2 §5.1 — `GET /system/config`

2.1 Add to `backend/app/api/v1/system.py`

```python
@router.get("/config")
async def get_system_config():
    """
    System configuration.
    Per MASTER_DOC_PART2 §5.1.
    Returns non-sensitive configuration values.
    """
    import os
    return {
        "capture": {
            "engine": "scapy",
            "features_per_flow": 63,
            "interface": os.environ.get("CAPTURE_INTERFACE", "eth0"),
        },
        "ml": {
            "ensemble_weights": {"isolation_forest": 0.30, "random_forest": 0.45,
"autoencoder": 0.25},
            "alert_thresholds": {"critical": 0.90, "high": 0.75, "medium": 0.50,
"low": 0.30},
            "dataset": "nsl_kdd",
            "scoring_mode": "ensemble",
        },
        "threat_intel": {
            "otx_enabled": bool(os.environ.get("OTX_API_KEY")),
            "abuseipdb_enabled": bool(os.environ.get("ABUSEIPDB_API_KEY")),
            "virustotal_enabled": bool(os.environ.get("VIRUSTOTAL_API_KEY")),
            "sync_interval_hours": 6,
        },
        "llm": {
            "provider": "openrouter",
            "models_count": 3,
        },
        "system": {
            "version": "0.4.0",
            "environment": os.environ.get("ENVIRONMENT", "production"),
            "dev_mode": os.environ.get("DEV_MODE", "false").lower() == "true",
        },
    }
```

Verification:

#	Check	Expected
1	`GET /system/config`	200, JSON with capture/ml/intel/llm config
2	Ensemble weights present	0.30/0.45/0.25
3	Thresholds present	0.90/0.75/0.50/0.30
4	No sensitive data exposed	No API keys in response
5	API coverage	42/42 (100%) ☑

TASK 3 — Alert IOC Enrichment in API Response 🟤

Time Est: 45 min | **Priority:** 🟤 Critical
Source: MASTER_DOC_PART4 §11.3 — Alerts should surface IOC match data

Currently, the `GET /alerts/{id}` endpoint returns alert data but does NOT include IOC correlation results. The frontend needs IOC match data to display threat context.

3.1 Update `backend/app/api/v1/alerts.py` — GET /alerts/{id}

After fetching the alert, run IOC correlation against the alert's source/dest IPs and include results:

```
@router.get("/{alert_id}")
async def get_alert(alert_id: str):
    """Get a single alert with IOC enrichment."""
    async with async_session() as session:
        result = await session.execute(
            text("""
                SELECT id, alert_id, severity, title, description,
                       category, source_ip, dest_ip, confidence, status,
                       ml_model, composite_score, if_score, rf_score, ae_score,
                       ai_narrative, created_at, updated_at
                FROM alerts
                WHERE alert_id = :aid OR id::text = :aid
            """),
            {"aid": alert_id},
        )
        row = result.fetchone()

    if not row:
        raise HTTPException(status_code=404, detail="Alert not found")

    alert_data = {
        "id": str(row[0]),
        "alert_id": row[1],
        "severity": row[2],
        "title": row[3],
        "description": row[4],
        "category": row[5],
        "source_ip": str(row[6]) if row[6] else None,
        "dest_ip": str(row[7]) if row[7] else None,
        "confidence": row[8],
        "status": row[9],
        "ml_model": row[10],
        "composite_score": row[11],
        "if_score": row[12],
        "rf_score": row[13],
        "ae_score": row[14],
        "ai_narrative": row[15],
        "created_at": str(row[16]),
        "updated_at": str(row[17]),
    }
```

```
    # IOC Enrichment — correlate alert IPs against IOC database
    ioc_enrichment = None
    try:
        from app.services.ioc_correlator import IOCCorrelator
        correlator = IOCCorrelator()
        ioc_result = await correlator.correlate_flow({
            "source_ip": alert_data["source_ip"],
            "dest_ip": alert_data["dest_ip"],
        })
        if ioc_result["has_ioc_match"]:
            ioc_enrichment = {
                "has_match": True,
                "src_match": ioc_result.get("src_match"),
                "dst_match": ioc_result.get("dst_match"),
                "domain_match": ioc_result.get("domain_match"),
                "flags": ioc_result.get("flags", []),
                "escalation": ioc_result.get("escalation_severity"),
            }
        else:
            ioc_enrichment = {"has_match": False}
    except Exception as e:
        logger.error("[Alerts] IOC enrichment failed: %s", e)
        ioc_enrichment = {"has_match": False, "error": str(e)}

    alert_data["ioc_enrichment"] = ioc_enrichment
    return alert_data
```

Verification:

#	Check	Expected
1	`GET /alerts/{id}` for alert with known IOC IP	`ioc_enrichment.has_match: true`
2	`ioc_enrichment.src_match` or `dst_match` populated	IOC data (threat_type, severity, source)
3	`GET /alerts/{id}` for alert without IOC match	`ioc_enrichment.has_match: false`
4	`ai_narrative` present in response	Markdown-formatted analyst report
5	ML scores present	composite_score, if_score, rf_score, ae_score
6	IOC enrichment does not slow down response	< 200ms total

TASK 4 — Execute IF Retrain with Tuned Params 🟢

Time Est: 30 min | **Priority:** 🟢 Medium
Source: MASTER_DOC_PART4 §4.4 — Tuned hyperparameters should produce new model artifact

The tuned IF parameters (`contamination=0.10`, `max_samples=1024`) were applied to `hyperparams.py` on Day 14, but the model file on disk (`isolation_forest.pkl`) was NOT retrained. Execute the retrain to

produce a new model artifact.

4.1 Trigger Retrain via API

```
curl -s -X POST http://localhost:8000/api/v1/ml/retrain \
  -H 'Content-Type: application/json' \
  -d '{"dataset": "nsl_kdd", "models": ["isolation_forest"]}' | python3 -m
json.tool
```

4.2 Monitor Retrain Task

```
# Get task_id from response, then poll:
curl -s http://localhost:8000/api/v1/ml/retrain/{task_id} | python3 -m json.tool
```

4.3 Verify New Model Performance

```
# After retrain completes:
docker compose exec ml-worker python -c "
from ml.inference.model_manager import ModelManager
mm = ModelManager()
mm.load_models()
print('Models loaded:', mm.get_status())
"
```

⚠ **CRITICAL: Ensemble weights (0.30/0.45/0.25) and thresholds remain LOCKED.**

Verification:

#	Check	Expected
1	`POST /ml/retrain` returns task_id	200, task_id present
2	`GET /ml/retrain/{task_id}` shows completion	`status: complete`
3	`isolation_forest.pkl` updated	File modified timestamp > Day 14
4	ML Worker loads new model	No errors in logs
5	Ensemble weights unchanged	0.30/0.45/0.25
6	Live inference works	New flows scored correctly
7	IF performance improved	Accuracy ≥82%, F1 ≥83% (from tuning results)

TASK 5 — Alert Cleanup (Old Test Data) ⬤

Time Est: 10 min | **Priority:** ⚪ Low
Source: SESSION_HANDOFF known issues

Remove seeded test alerts from Day 12 that clutter the alert table:

```
-- Run in postgres container:
DELETE FROM alerts WHERE alert_id LIKE 'TM-ALERT-%';
-- This removes the old format alerts (TM-ALERT-00002..00006)
-- New format alerts (TM-YYYYMMDDHHMMSS-XXXXXXXX) are preserved
```

Verification:

#	Check	Expected
1	`SELECT COUNT(*) FROM alerts WHERE alert_id LIKE 'TM-ALERT-%'`	0
2	`SELECT COUNT(*) FROM alerts WHERE alert_id LIKE 'TM-20%'`	3+ (real alerts preserved)
3	`GET /alerts/`	Only real ML-generated alerts

TASK 6 — CICIDS2017 Validation (Optional) ⚪

Time Est: 60 min | **Priority:** ⚪ Low — Academic credibility enhancement
Source: MASTER_DOC_PART4 §3, MASTER_DOC_PART5 §10.3

Download and validate models against the CICIDS2017 dataset for academic diversity.

6.1 Dataset Info

```
URL: https://www.unb.ca/cic/datasets/ids-2017.html
Size: ~6.5 GB (full), can use subset
Format: CSV with labeled network flows
Labels: BENIGN, DoS, PortScan, DDoS, Bot, Infiltration, etc.
```

6.2 Create CICIDS2017 Loader

Create `backend/ml/datasets/cicids2017.py`:

```python
"""CICIDS2017 dataset loader for model validation."""

import pandas as pd
from pathlib import Path

def load_cicids2017(data_dir: str = "ml/datasets/cicids2017/"):
    """Load and preprocess CICIDS2017 CSV files."""
```

```
    # Map CICIDS features to our 63-feature schema
    # Validate ensemble against second dataset
    # Report accuracy, F1, AUC-ROC
    ...
```

This task is OPTIONAL for Day 15. If time permits, implement it. Otherwise defer to Week 5 per timeline.

Verification:

#	Check	Expected
1	Dataset downloaded	CSV files present in ml/datasets/cicids2017/
2	Loader runs without errors	DataFrame with features + labels
3	Ensemble scores CICIDS2017 flows	Accuracy/F1/AUC-ROC reported
4	Results saved	JSON report in eval_results/

Post-Task API Coverage Update

Service	Before	After	Coverage
Auth	5/5	5/5	**100%**
Flows	6/6	6/6	**100%**
Alerts	5/5	5/5	**100%**
Capture	5/5	5/5	**100%**
System	2/3	**3/3**	**100%** ☑ (+config)
WebSocket	1/1	1/1	**100%**
ML	5/5	5/5	**100%**
LLM	5/5	5/5	**100%**
Intel	4/4	4/4	**100%**
Reports	0/3	**3/3**	**100%** ☑ (+generate, list, download)
TOTAL	**38/42**	**42/42**	**100%** 🎯

Files Modified / Created (Expected)

File	Action	Lines (est.)
`app/api/v1/reports.py`	CREATE	~180 (3 endpoints)
`app/api/v1/system.py`	MODIFY	+25 (config endpoint)
`app/api/v1/alerts.py`	MODIFY	+30 (IOC enrichment)

File	Action	Lines (est.)
`app/main.py`	MODIFY	+2 (reports router)

Risk Assessment

Risk	Impact	Mitigation
system_config table used for reports storage	Schema coupling	Acceptable for MVP, dedicated reports table in Week 6
PDF not generated (JSON only)	Demo impact	Part 3 §10 allows PDF in Week 6 (ReportLab)
IF retrain takes too long	Blocks verification	Set timeout, fallback to existing model
IOC enrichment adds latency to alert API	Performance	Cache correlator results, keep async

---

STRICT RULES REMINDER

1. **DO NOT** change ensemble weights (0.30/0.45/0.25) — LOCKED
2. **DO NOT** change alert thresholds (0.90/0.75/0.50/0.30) — LOCKED
3. **DO NOT** suggest Kafka, Kubernetes, Elasticsearch, MongoDB
4. **DO NOT** add features not in the 10 modules
5. **DO NOT** use Tailwind CSS — Vanilla CSS + CSS Variables only
6. LLM via **OpenRouter only** — 3 verified models
7. Prompts follow **PART4 §9.2** templates
8. Master documentation (5 parts) is the **SOLE source of truth**
9. All code: **typed, error-handled, documented, production-quality**
10. Python: **type hints, async/await, SQLAlchemy 2.x**

Day 15 Worklog — Week 4 Day 2
v0.4.0 Critical MVP: ACHIEVED ☑
E2E Pipeline LIVE: capture → ML → alerts → IOC (IP+domain+hash) → LLM narrative → WebSocket
Target: 42/42 API coverage (100%) + Alert IOC enrichment + IF retrain + Reports module
Ensemble: 80.73% acc | 80.96% F1 | 0.9312 AUC-ROC (LOCKED)
IOC Database: 1,367 indicators (§11.3 fully compliant)